TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

UNIVERSITÀ DEGLI STUDI DI ROMA "TOR VERGATA"
CORSO DI LAUREA IN INFORMATICA TRIENNALE

BlueAgent

Orchestratore di container per Cyber Range orientati a CTF di Difesa



Candidato: Davide Luci
Matricola: 0341922

Relatore: Prof. Pierpaolo Loreti
Correlatore: Ing. Francesco Mancini

BlueAgent: Orchestratore per CTF di Difesa

Contesto e Obiettivi

Contesto: gestione di challenge CTF orientate alla Difesa (la flag si ottiene *patchando* e mettendo in sicurezza la vulnerabilità).

Requisiti chiave:

1. Gestire una o più macchine comunicanti (scenari multi-nodo).
2. Determinare lo stato delle macchine all'oscuro dei giocatori.
3. Preservare l'operatività dei servizi legittimi dopo le patch.

Soluzione e Implementazione

La soluzione, BlueAgent: Orchestratore su singolo host, scritto in **Python** e basato sul framework **FastAPI**.

- Gestisce **system container (Incus).**
- Verifica le patch clonando i container in un ambiente di rete inaccessibile ai giocatori.
- **Validazione:** misurazione dei tempi di esecuzione e dell'uso di CPU e RAM al crescere di team e container.

Cyber Range Alternativi: kCTF e CTFBox

kCTF (Google)

- Basato su cluster **Kubernetes** e **NSJail**.
- Un unico container condiviso: ad ogni connessione esegue un fork e rilascia un ambiente separato.
- Usa seccomp, Namespaces, cgroups e fornisce un filesystem in RAM (`tmpfs`).
- **Limite:** assenza di `systemd` e impossibilità di installare pacchetti o modificare servizi permanentemente.

CTFBox

- Pensato per competizioni **Attack & Defense** competitive tra squadre. Basato su Docker, Incus e WireGuard.
- I Checkers interrogano i servizi ad ogni tick su rete condivisa. Calcolano uptime, SLA ed installano nuove flag.
- 3 stati di rete: freeze, lock e unlock.
- **Limite:** assenza di un oracolo isolato; il traffico di verifica è visibile ai team sulla rete di gioco.

Cyber Range Alternativi: CyTrONE e CyberRangeCZ

CyTrONE

- Integra una piattaforma didattica (LMS) ed un Cyber Range. Basato su **Macchine Virtuali KVM**.
- Supporta training di difesa definiti tramite YAML.
- Un controller invia exploit via rete verso le macchine degli studenti.
- Usa `parallel-scp` per distribuire le immagini.

CyberRangeCZ / KYPO

- Piattaforma cloud basata su **OpenStack**, Kubernetes, Terraform e Ansible.
- Sandbox di Macchine Virtuali per emulare scenari.
- Accesso dei partecipanti regolato da un gateway (MAN).
- Requisiti elevati per gestire centinaia di VM.

Cyber Range Alternativi: CyberPatriot

Architettura

- Programma educativo nazionale (AFA) basato su **Macchine Virtuali.**
- I partecipanti operano come amministratori di sistemi vulnerabili.

Limiti rispetto a BlueAgent

- **Consumo elevato:** diversi GB di RAM per singola VM (contro ~50 MB in BlueAgent).
- **CSS:** lo scoring avviene tramite un demone residente all'interno della VM.
- Piattaforma **closed-source.**

Confronto con lo Stato dell'Arte

Caratteristica	kCTF	CTFBox	CyTrONE	CyberRangeCZ	CyberPatriot	BlueAgent
Componenti	Kubernetes, NSJail	VM, Container di sistema (Incus)	Macchine Virtuali	Macchine Virtuali	Macchine Virtuali	Container di sistema (Incus)
Infrastruttura	Cluster Kubernetes	Singolo host	Cluster distribuito	Cloud OpenStack	N/A	Singolo nodo host
Rete / Ambiente	Condiviso (`tmpfs`)	Subnet dedicata ai team condivisa	Reti KVM	Rete OpenStack	VM dedicata	LAN isolata per team
Verifica	Container Healthcheck	Checkers ad ogni tick	Exploit via rete	Non prevista	Demone interno	Clonazione
Invisibilità Verifica	**Visibile**	**Assente**	**Visibile**	**Assente**	**Visibile**	**Invisibile**

Attori del Sistema

CTF Manager

- Unica entità autenticata che comunica direttamente con BlueAgent tramite **API REST** (autenticate con token condiviso).
- Mantiene lo stato della competizione (punteggi, squadre, progressione), disaccoppiando la gara dall'infrastruttura.
- Invia a BlueAgent le configurazioni YAML e inoltra le richieste di verifica.

Attori Esterni (tramite CTF Manager)

- **Giocatore:** richiede il deploy dell'ambiente, ottiene la configurazione WG e richiede le verifiche.
- **Autore della sfida:** scrive i file YAML della topologia e le definizioni Distrobuilder.
- **Amministratore:** monitora lo stato dei team, l'infrastruttura e le risorse attive.
- Nessun attore esterno comunica direttamente con BlueAgent o ha accesso alle sue API.

Use Cases Principali

UC-01 Setup Rete Team

`PUT /teams/{team_k}`

Alloca il bridge di produzione `br-team-k` (`10.k.1.1/24`) e il gateway WireGuard `wg-k` associandogli la porta $51820 + k$.

UC-02 Deploy Challenge

`POST /teams/{team_k}/machines`

Istanzia i container di sistema, collega `eth0` a `br-team-k` , inietta la configurazione IP statica e applica i limiti cgroups.

UC-03 Verifica Challenge

`POST /teams/{team_k}/verificator`

Crea una rete OVN separata, clona i container con ZFS, esegue i comandi di readiness e i comandi di verifica dell'oracolo.

UC-06 Accesso VPN Membro

`PUT /teams/{team_k}/members/{user_id}`

Alloca un IP libero (`10.k.0.x`), genera le chiavi crittografiche, registra il peer e restituisce il file `.conf` .

Requisiti di Sistema Rappresentativi

Infrastruttura e Rete

- **Host e Storage:** Debian 13 con `nftables` e storage pool ZFS.
- **Una rete per Team:** tunnel `wg-k` e bridge `br-team-k` comunicanti, isolati per ogni squadra k.
- **Cgroups:** applicazione di limiti su CPU e memoria RAM per impedire starvation.
- **TCP MSS Clamping:** forzatura dell'MSS in nftables per evitare MTU blackhole.

Verifica e Control Plane

- **Fedeltà Topologica:** i cloni preservano l'interfaccia `eth0` e l'IP (`10.k.1.x`) di produzione.
- **Zero Side-Channel:** traffico e container di verifica confinati nella rete OVN, inaccessibili ai giocatori.
- **Affidabilità:** mutua esclusione con `TwoTierLock` e rilascio di risorse inutilizzate.
- **Lifecycle:** demone per la pulizia delle risorse orfane.

L'Oracolo e i Meccanismi di Verifica

Oracolo (`OVNVerifier`): determina se il sistema è *patchato* eseguendo comandi sui nodi e confrontando l'output con il valore atteso.

Approcci Tradizionali (Scartati)

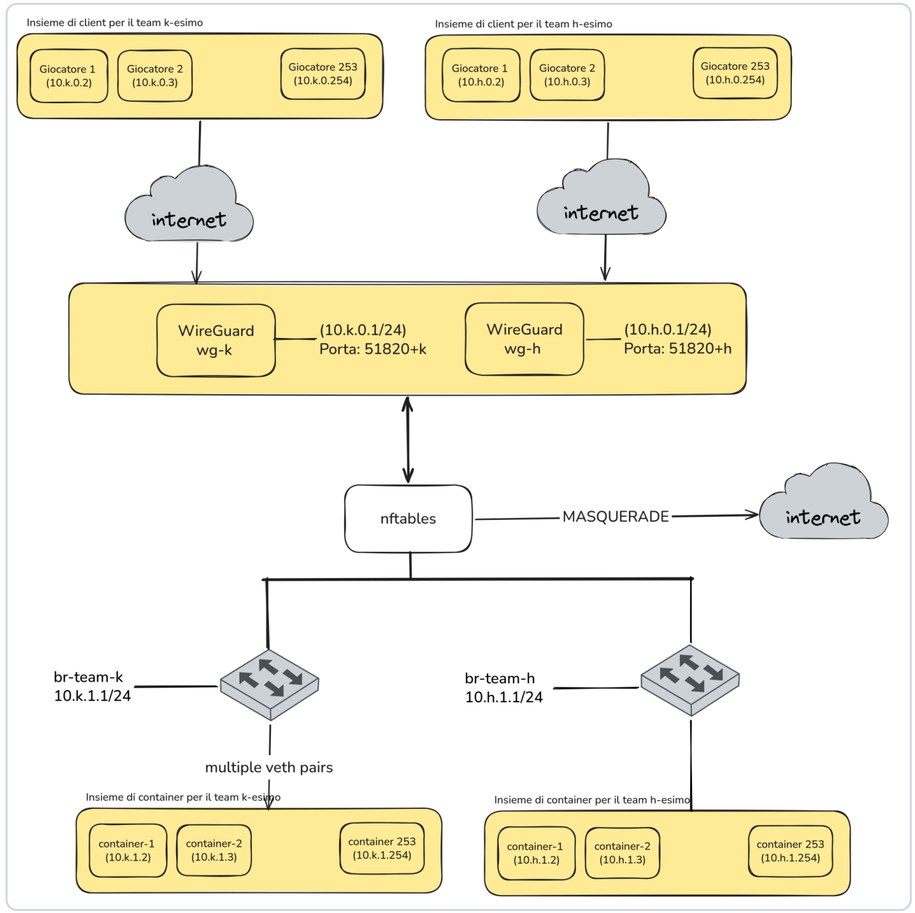
- **Verifica Statica (diff, SAST):** falsi positivi e inefficace sui binari compilati.
- **Verifica Dinamica esterna:** i giocatori (root) intercettano i payload con `tcpdump`.
- **Injection (`incus exec`):** i giocatori intercettano i comandi con `pspy` su `/proc`.

Approccio BlueAgent

- **Clonazione in rete separata:** duplica i container dentro una rete OVN inaccessibile ai giocatori.
- Impossibile intercettare il traffico di rete.
- Il giocatore non può accedere con ssh al container di verifica.

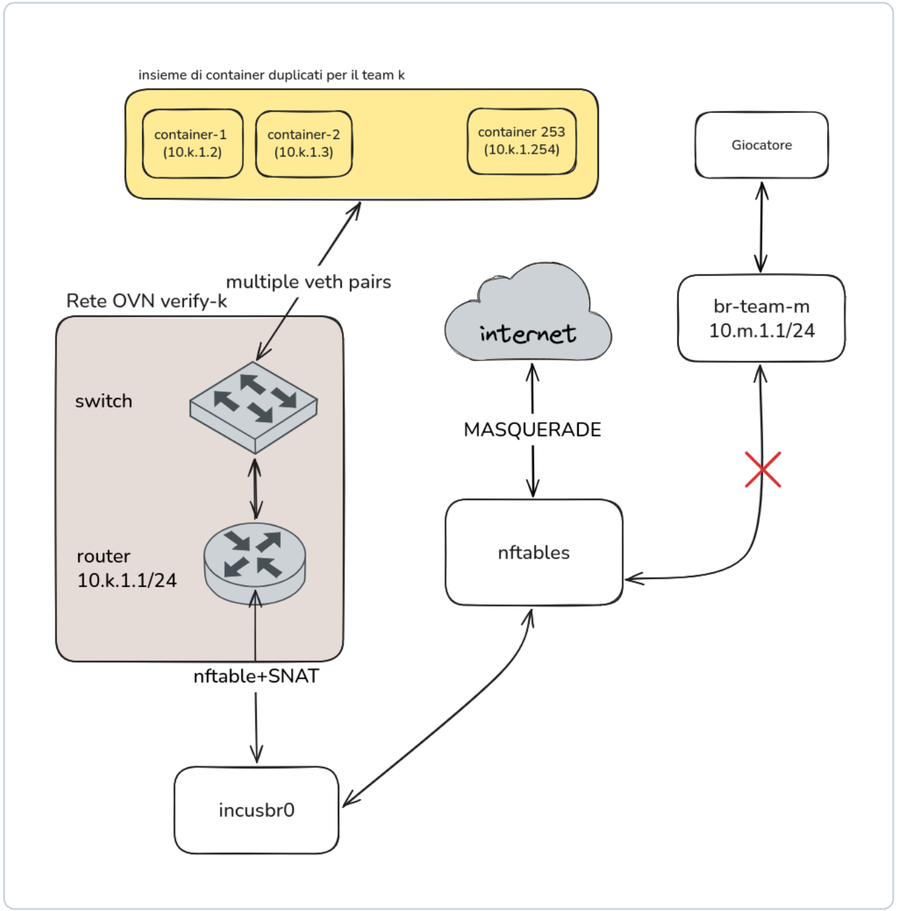
Rete production-k

- `wg-k` **(Gateway VPN):** terminazione WireGuard dedicata al team k (`10.k.0.0/24` , porta UDP $51820 + k$).
- `br-team-k` **e** `veth pair` **:** bridge Linux per la LAN dei container (`10.k.1.1/24`), collegato a `eth0` tramite coppie virtuali `veth` .
- **Isolamento tra Team:** policy di *drop* predefinita in `nftables` ; **team diversi non possono comunicare tra loro.**
- **Set** `@team_links` **:** autorizza il forwarding bidirezionale esclusivamente tra l'interfaccia `wg-k` e il rispettivo bridge `br-team-k.`



Rete verify-k: SDN, OVN e OpenFlow

- **Sandbox SDN Isolata:** rete virtuale OVN istanziata ad ogni verifica, separata da `production-k` e dalle altre squadre.
- **Logical Switch e Router OVN:** replicano la LAN (`10.k.1.0/24`) e il gateway (`10.k.1.1/24`) per i container clonati, senza conflitti di routing sull'host.
- `incusbr0` (`172.16.0.1/24`)**:** bridge di transito condiviso che collega i router OVN verso la WAN.
- **Piano di Controllo (OVN):** traduce la topologia logica in regole di matching e azioni **OpenFlow**.
- **Piano dei Dati (Open vSwitch):** applica i flussi OpenFlow nel kernel per inoltrare i pacchetti tra i cloni.



Meccanismo di SNAT a Due Stadi

Evita collisioni di routing sull'host nascondendo gli IP duplicati (`10.k.1.x`) dei container di verifica:

- **Primo Stadio (SNAT in OVN):** il Router riscrive l'IP src con l'IP su `incusbr0` (`172.16.0.x`).
- **Secondo Stadio (SNAT su `incusbr0`):** `nftables` applica il *masquerade* con l'IP pubblico dell'host.

Tratta del Pacchetto	Indirizzo Sorgente (IP_{src})	Indirizzo Destinazione (IP_{dst})	Responsabile
Dal Container ad `incusbr0`	`10.k.1.x` *(IP duplicato)*	IP_{ext} *(Internet)*	Container di verifica
Da `incusbr0` all'interfaccia dell'host	`172.16.0.x` *(IP transito univoco)*	IP_{ext} *(Internet)*	Incus (OVN)
Dall'host verso l'esterno	IP dell'interfaccia pubblica dell'host	IP_{ext} *(Internet)*	`nftables`

Struttura YAML della Challenge

Topologia e Metadati

- **Metadati:** `challenge_id`, `team_k`.
- `containers`**:** lista delle macchine. Si specificano `image`, `ip_suffix` (`10.k.1.X`) e limiti CPU/RAM.
- `verifier` **(opzionale):** container dedicato ai test nella rete OVN (`10.k.1.250`).

Comandi di Verifica

- `readiness_commands`**:** attende l'avvio dei servizi prima dei test (con retry e timeout).
- `verify_commands`**:**
  - `target: "container"` — eseguito dentro il clone con `incus exec`.
  - `target: "verifier"` — eseguito dentro il container di verifica con `incus exec`.

Configurazione Immagini con Distrobuilder

Definizione Dichiarativa

Distrobuilder: tool ufficiale per generare immagini di container Incus in modo riproducibile.

- `image` e `source`: distribuzione Linux di base (es. Ubuntu 22.04, Alpine 3.21).
- `packages`: pacchetti software da includere nel rootfs (Nginx, PostgreSQL, Python).

Predisposizione Rete e Import

- `actions`: uso di hook per predisporre i servizi di rete (`systemd-networkd` o `/etc/network`) ed installare le configurazioni in `/etc`.
- **Pool ZFS:** l'archivio unificato prodotto viene importato nella storage pool ZFS.

Architettura del Codice

Strati Applicativi

- `/api`: endpoint REST scritti in FastAPI, autenticati via token.
- `/services`: logica di business dell'orchestrazione.
- **Tecnologie e librerie:** `FastAPI`, `Pydantic`, `httpx`, `uvicorn` e `SQLite`.

Driver e Moduli Specializzati

- `/engine`: interfaccia astratta verso il runtime Incus.
- `/network`: gestione di bridge Linux, WireGuard e regole `nftables`.
- `/verifier`: coordinamento della rete OVN e dei comandi dell'oracolo.
- `/images` e `/lifecycle`: compilazione rootfs e pulizia risorse orfane.

Astrazione del Runtime: `ContainerEngine`

Interfaccia Astratta

Disaccoppia il servizio dall'implementazione specifica del demone container.

Primitive esposte:

- Creazione, avvio, arresto ed eliminazione istanze.
- Duplicazione rapida e configurazione schede di rete.
- Esecuzione non bloccante di comandi nei container.

Implementazione `IncusEngine`

- **System Container:** offre un gestore di servizi (ed un suo PID 1) nel container e l'infrastruttura per un sistema realistico.
- Comunicazione tramite **Unix Domain Socket** (`/var/lib/incus/unix.socket`): dati trasferiti direttamente in memoria kernel.

Async Context Manager in `OVNVerifier`

Implementa il pattern **RAII** per garantire che la sandbox esista solo per la durata della verifica:

1. `__aenter__` e 2. `run_checks`

- Crea la rete OVN temporanea.
- Clona i container tramite snapshot ZFS e collega ogni `eth0` a OVN.
- Avvia il *Verifier Runner* ed esegue i test.

3. Teardown Sicuro (`__aexit__`)

1. Arresta ed elimina il container `verify-runner` .
2. Arresta ed elimina tutti i container clonati.
3. Elimina la rete virtuale OVN.

Garantito in caso di errore o eccezione.

Gestione della Concorrenza: `TwoTierLock`

I Due Livelli di Lock

- **Tier 1 (Intra-processo):** usa `asyncio.Lock` tra task concorrenti sullo stesso worker.
- **Tier 2 (Inter-processo):** usa `fcntl.flock` sul filesystem tra più processi worker (`uvicorn`).
- Garantisce coerenza anche in architetture multi-worker (es. `--workers 4`).

Ambito di Applicazione

- **Per Team:** mutua esclusione su tutte le operazioni del team k.
- **Per Istanza e Globale:** protegge i singoli container e le modifiche a `nftables` .
- **Rientrante:** uno stesso task può acquisire più volte il lock tramite contatore.

Pipeline Asincrona, Lifecycle e Immagini

Pipeline Asincrona Non Bloccante

- Client HTTP asincrono (`httpx.AsyncClient`) sul socket Unix di Incus.
- Attesa non bloccante delle operazioni su `/1.0/operations/<uuid>/wait` .
- Comandi host eseguiti tramite `asyncio.create_subprocess_exec` .

`LifecycleManager` e Immagini

- `LifecycleManager` : demone che rimuove container e reti di verifica rimasti attivi oltre il *grace period.*
- **Caching SHA-256:** calcola l'hash dello YAML dell'immagine per evitare ricompilazioni inutili.
- Log delle build accessibili in streaming via API.

Struttura del Plugin CTFd

Componenti Estese

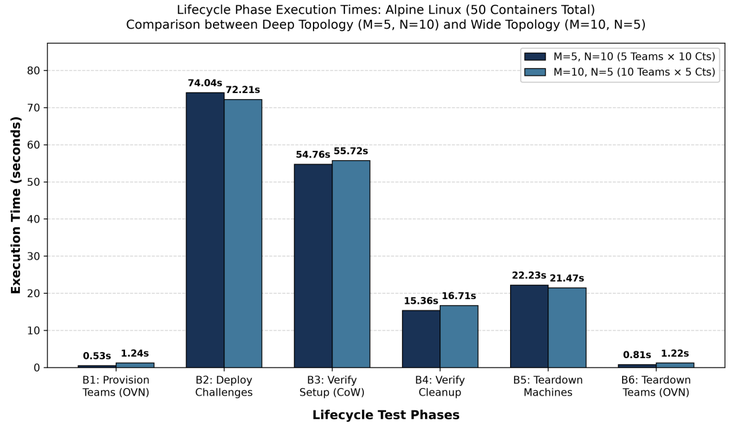
- `PatchChallenge`: memorizza nel database di CTFd la configurazione YAML delle challenge.
- `PatchFlag`: ridefinisce `compare()` interrogando l'Oracolo (BlueAgent).
- Uso di SQLAlchemy di CTFd per definire il modello dei dati.

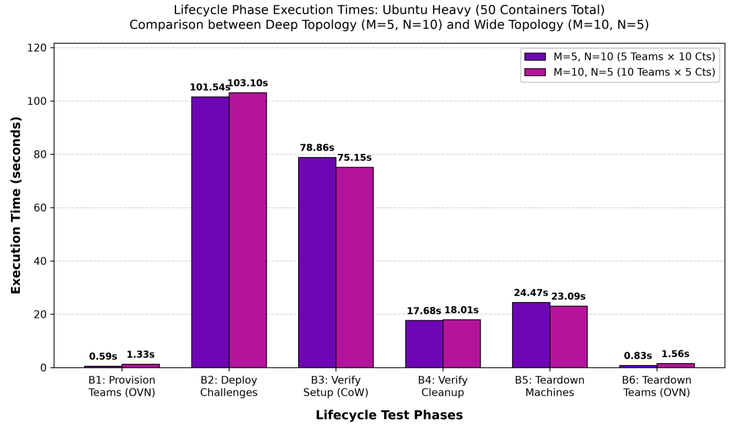
Flusso Operativo

- **Accesso VPN:** richiede a BlueAgent la creazione del peer e consegna al giocatore il file WireGuard `.conf`.
- **Verifica:** inoltra la richiesta all'Oracolo e aggiorna il punteggio se la patch viene convalidata.
- Visualizzazione dello stato dei container nella dashboard giocatore.

Validazione: Confronto tra Fasi (Alpine vs Ubuntu)

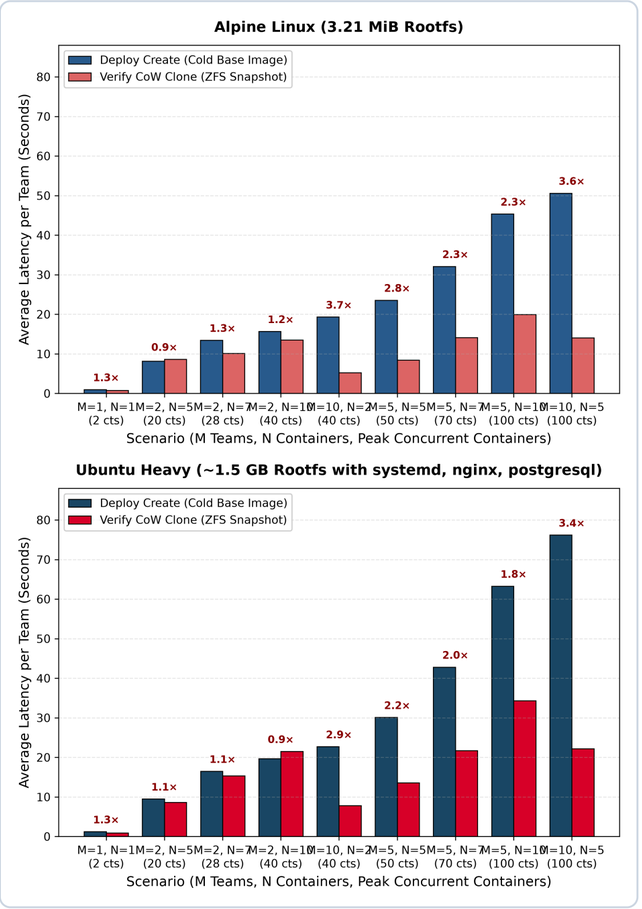
- **Piattaforma:** 8 Core Ryzen 7 7700X, 16 GB RAM, SSD ZFS.
- **Immagini:** `alpine` ($3.2\ \mathrm{MB}$) e `ubuntu-heavy` ($\sim 1.5\ \mathrm{GB}$).
- $T(B_3) < T(B_2)$: la verifica (B_3) è sempre più veloce del deploy iniziale (B_2).
- Nei casi (5,10) e (10,5) si risparmiano **~20s** su Alpine e **>25s** su Ubuntu Heavy.
- **Vantaggio della Clonazione:** nella verifica (B_3) il container è già esistente; BlueAgent esegue uno snapshot ZFS ed un clone CoW.





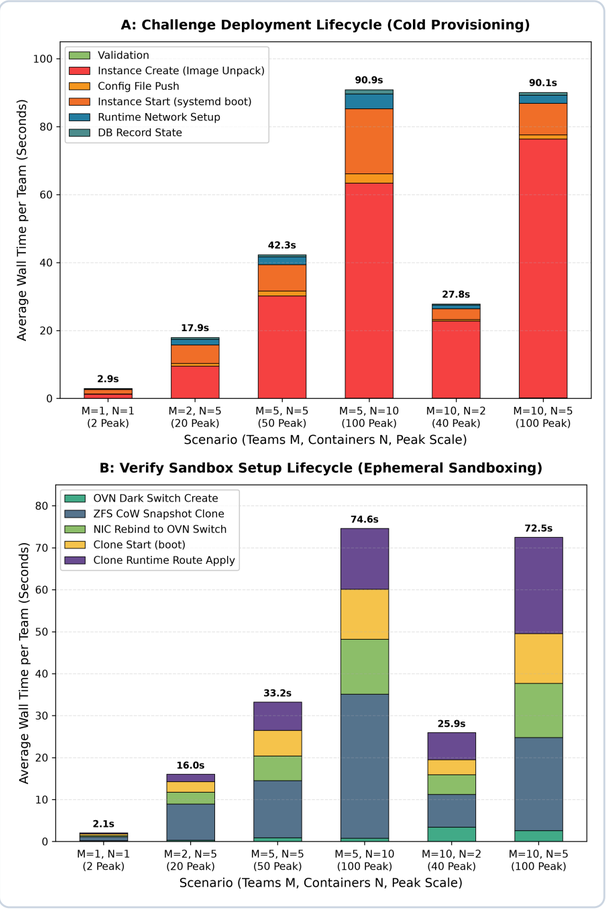
Validazione: Clonazione CoW vs Deploy

- `Deploy Create`: tempo di creazione a freddo; cresce sensibilmente con il numero di container e la dimensione dell'immagine.
- `Verify CoW Clone`: tempo di clonazione istantaneo; resta nettamente inferiore (Speedup fino a **3.6x** nello scenario 10×5).
- **Parallelismo CPU:** clonare i container in $(10, 5)$ è più rapido di $(5, 10)$ sfruttando 10 task paralleli:
  - In $(5, 10)$, 5 worker devono smaltire una coda più lunga di operazioni concorrenti.



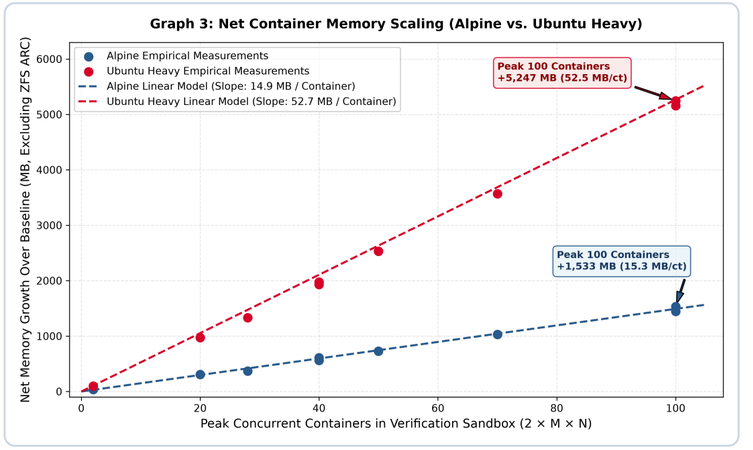
Validazione: Ripartizione nelle Sotto-Fasi

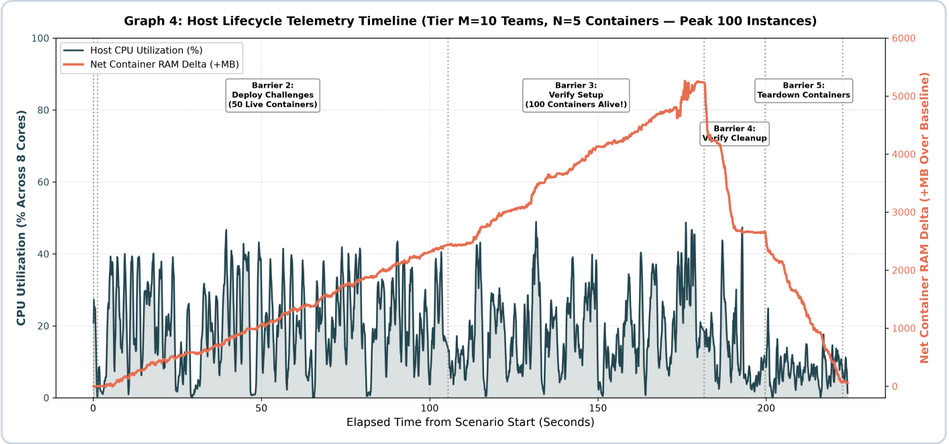
- **Deploy a Freddo (Grafico A):** dominato da *Instance Create* (unpack dell'immagine rootfs da disco, fino all'80% del tempo totale).
- **Verifica Sandbox (Grafico B):** la clonazione CoW ZFS sostituisce Instance Create, abbattendo la creazione del filesystem.
- **Fasi di Verifica:** il tempo residuo è ripartito tra boot dei servizi (*systemd*) e applicazione dei flussi di rete OVN.
- **Sviluppi Futuri:** istanziare preventivamente ogni challenge (pre-allocate) per velocizzare anche il deploy iniziale.



Validazione: Uso di Memoria RAM e CPU

- **100 container Ubuntu Heavy** richiedono solo **~5.2 GB** di RAM netta dell'host.
- Consumo medio RAM container:
  - **Ubuntu Heavy:** $\sim 52.5\ \mathrm{MB}$ per container.
  - **Alpine:** $\sim 15.3\ \mathrm{MB}$ per container.
- **Densità Elevata:** e' possibile tenere in memoria un gran numero di container con macchine da 16GB o 32GB di RAM.
- **CPU & I/O:** la CPU non satura mai l'host; il collo di bottiglia principale è limitato alle transazioni di I/O su storage.





Architetture di Rete Alternative: 6.1 e 6.2

6.1 Bridge L2 Senza Gateway

- Collega i container clonati a un semplice bridge L2 privo di indirizzo gateway.
- **Pro:** implementazione minima e zero conflitti IP.
- **Perché scartata:** nessun accesso a Internet durante la verifica (comandi come `apt` falliscono).

6.2 Interfaccia `wan1` Aggiuntiva

- Aggiunge a ogni clone una seconda interfaccia `wan1` collegata a `incusbr0`.
- **Pro:** fornisce accesso a Internet senza gateway sul bridge.
- **Perché scartata:** altera le tabelle di routing dei container e raddoppia le coppie `veth`.

Architetture di Rete Alternative: 6.3 e 6.4

6.3 Network Namespace Separato

- Crea il bridge di verifica (con gateway `10.k.1.1/24`) dentro un nuovo Network Namespace collegato via `veth`.
- **Perché scartata:** Incus opera nel namespace root e non gestisce namespace esterni, richiedendo una gestione manuale complessa.

6.4 Router Intermedio

- Usa un container Linux dedicato come gateway (`10.k.1.1/24`) che applica SNAT verso `incusbr0`.
- **Perché scartata:** richiede di orchestrare un container aggiuntivo per ogni verifica, con maggiore complessità rispetto a OVN.

Tabella Riassuntiva delle Architetture di Verifica

Architettura	Fedeltà di Rete	Accesso WAN	Complessità Software	Overhead Risorse
6.1 Bridge L2 (senza GW)	Bassa (nessun default gateway)	Assente	Minima (bridge kernel)	Quasi nullo
6.2 Interfaccia `wan1`	Scadente (interfaccia anomala)	Presente (tramite `wan1`)	Media (routing custom container)	Basso
6.3 NetNS separato	Buona (topologia speculare)	Presente (coppia `veth`)	Elevata (gestione manuale `veth`)	Basso
6.4 Router intermedio	Buona (topologia identica)	Presente (SNAT su router)	Medio-alta (container dedicato)	Medio (+1 container)
BlueAgent (OVN SDN)	**Buona (topologia identica)**	**Presente (SNAT 2 stadi)**	**Minima (delegata a Incus)**	**Basso (condiviso)**

Difesa Architetturale, Compromessi e Sviluppi Futuri

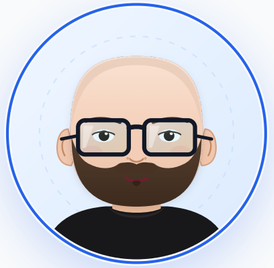
Difesa e Compromessi

- **Vantaggi:** Debian 13 stabile, ZFS con CoW, container Incus che offrono `init` e invisibilità totale grazie ad OVN.
- **Compromessi:**
  - *UDP MTU Blackhole:* scarto pacchetti UDP a 1500B con DF su tunnel WireGuard.
  - *Scalabilità:* max 254 team (`10.k.x.x`) e locking su singolo host.

Sviluppi Futuri

- **CLI `bluecmd` :** gestione e debug in locale senza dipendere da CTFd.
- **Monitoraggio SLA:** controlli periodici sui container di produzione per penalizzare chi spegne i servizi.
- **YAML Avanzato:** supporto a topologie multi-tier con router dedicati.

Conclusioni e Spazio alle Domande



Davide Luci

Matricola: 0341922

Informatica Triennale — Tor Vergata

☆ Riepilogo Contributi di Tesi

- Orchestrazione leggera di system container con **Incus**.
- Isolamento trasparente della verifica mediante **OVN & OpenFlow**.
- Clonazione dei container tramite snapshot **ZFS**.
- Integrazione di oracolo e plugin di verifica automatica in **CTFd**.